

DESIGN DOCUMENT

PostgreSQL DBA Auditor MCP Server

A practical MCP server for understanding, monitoring, and maintaining PostgreSQL databases

Version	Status	Author	Reviewed By
1.0 — DRAFT	Under Review	Platform Engineering	TBD

1. Problem Statement

When something goes wrong in a PostgreSQL database — queries slow down, connections pile up, disk usage spikes — the standard response is to open a terminal, run a handful of system queries from memory or StackOverflow, and try to piece together what's happening from raw numbers across half a dozen views.

This takes time, requires expertise, and doesn't scale. A senior DBA knows exactly where to look: `pg_stat_statements` for slow queries, `pg_stat_activity` for what sessions are doing right now, `pg_stat_user_tables` for bloat and vacuum status, `pg_locks` for contention. But that expertise is scarce, and most teams don't have a dedicated DBA on call.

LLMs are good at exactly this kind of structured investigation — reading data, spotting patterns, explaining what they mean, and recommending actions. The gap is that there's no clean, safe way for an LLM agent to query these system views, act on what it finds (cancel a blocking query, kick off a vacuum), and do all of this without ever touching application data.

What we're building: An MCP server that gives an AI agent the same system-level view a DBA would use — slow queries, active sessions, lock chains, cache hit rates, bloat, index health — plus the ability to take safe, audited maintenance actions like `VACUUM` and session termination.

2. What Already Exists

Three open-source PostgreSQL MCP servers exist today. We reviewed each before designing this server.

2.1 crystalldba/postgres-mcp (Postgres MCP Pro)

GitHub: github.com/crystalldba/postgres-mcp | Language: Python | Stars: ~2,000

The most purpose-built of the three. Postgres MCP Pro focuses specifically on health analysis, query performance, and index tuning. It exposes 9 clean tools, runs in either restricted (read-only) or unrestricted mode, and uses hypothetical indexes via HypoPG to simulate the impact of indexes before creating them.

What it covers	Detail
Schema exploration	List schemas, objects, and details about tables/indexes/constraints
SQL execution	Run queries — read-only in restricted mode, full access in unrestricted
Index analysis	Recommends indexes based on workload (pg_stat_statements) using a cost-based DTA algorithm or LLM-based method. Uses HypoPG to predict improvement without building the real index
Query plan analysis	EXPLAIN with optional ANALYZE; supports hypothetical index testing
Database health	Single analyze_db_health tool covering index, vacuum, connection, buffer, replication, sequence, and constraint health
Top queries	Returns slowest or most resource-intensive queries from pg_stat_statements

What it is missing: No active session management (cannot see who is blocking whom, cannot cancel or terminate sessions). No VACUUM or REINDEX execution. No per-table cache hit rates. No lock tree visualization. No maintenance progress monitoring. Development has slowed — last commit January 2026, 25 open issues.

2.2 pgEdge/pgedge-postgres-mcp

GitHub: github.com/pgEdge/pgedge-postgres-mcp | Language: Go | Stars: ~99 | Status: Beta

A natural language agent for PostgreSQL — its main purpose is translating plain English questions into SQL and running them. It ships with a CLI and a Web UI, supports PostgreSQL 14+, and is designed for internal developer tools where all users are trusted.

What it covers	Detail
----------------	--------

Natural language to SQL	LLM translates user questions into SQL; the server executes them
Read-only query execution	All queries run in read-only transactions by default
Schema introspection	get_schema_info returns schema metadata in TSV format
Resource access	Exposes PostgreSQL statistics as MCP resources
Custom tools	Write PL/pgSQL in a YAML file to define custom tools — extensible by operators
Web UI	Browser-based interface for querying without a terminal

What it is missing: Focused on query execution, not DBA observability or maintenance. No slow query profiling, no index recommendations, no vacuum or bloat analysis, no session management, no lock detection. Explicitly warns it is not for production use without thorough testing.

2.3 neverinfamous/postgres-mcp

GitHub: github.com/neverinfamous/postgres-mcp | Language: Node.js | Stars: ~7

The most feature-rich of the three — 248 specialized tools, 23 resources, 20 prompts, and a "Code Mode" that replaces all tools with a single V8 sandbox to reduce token usage by up to 90%. Covers schema management, migration tracking, DDL generation, backups, vector search, geospatial operations, and more. Supports 12 PostgreSQL extensions.

What it covers	Detail
Schema & DDL	Full schema management, DDL generation, migration tracking and execution
CRUD operations	Insert, update, delete, upsert, bulk operations on application tables
Health & observability	Comprehensive health check covering cache hit ratios, bloat, connections, replication
Index analysis	Workload-based index recommendations using pg_stat_statements
Performance monitoring	Top queries, slow query identification
Extensions	pgvector, PostGIS, pg_partman, pg_cron, pg_stat_kcache, HypoPG, and more
Auth & transport	OAuth 2.1, HTTP/SSE, tool filtering presets, audit logging
Code Mode	Single sandboxed tool replacing all 248 — reduces token overhead significantly

What it is missing: Despite broad coverage, it does not expose active session management (no cancel/terminate), no live lock tree, no VACUUM or REINDEX execution, and no maintenance progress monitoring. Its breadth also makes it harder to

deploy in security-sensitive environments where restricting to DBA-specific operations is required.

2.4 Gap Summary — What None of Them Do

Capability	crystaldba	pgEdge	neverinfamous
Slow query identification	✓ Yes	✗ No	✓ Yes
Index recommendations	✓ Yes	✗ No	✓ Yes
EXPLAIN plan analysis	✓ Yes	✗ No	~ Partial
Database health overview	✓ Yes	✗ No	✓ Yes
Cache hit rate (per table)	~ Partial	✗ No	~ Partial
Active session view	✗ No	✗ No	✗ No
Lock tree / blocking chains	✗ No	✗ No	✗ No
Cancel or terminate a session	✗ No	✗ No	✗ No
VACUUM table execution	✗ No	✗ No	✗ No
REINDEX execution	✗ No	✗ No	✗ No
Vacuum / REINDEX progress	✗ No	✗ No	✗ No
Autovacuum health per table	✓ Yes	✗ No	~ Partial
Maintenance impact estimate	✗ No	✗ No	✗ No
Read-only by default (no app data)	✓ Yes	✓ Yes	✗ No

Our focus: This server fills the gaps highlighted above — active session management, lock visibility, maintenance operations (VACUUM, REINDEX), and deeper observability — while staying firmly in the DBA/system layer. It never reads or writes application data.

3. Goals & Non-Goals

3.1 Goals

- Show slow queries — what is running slowly, how often, and how much resource it uses.

- Explain query plans — run EXPLAIN and describe in plain language what the planner is doing and why it might be slow.
- Recommend indexes — identify missing indexes, flag unused ones, detect duplicates and bloated indexes.
- Show cache hit rates — at database level and per table, flag where hit rates are below acceptable thresholds.
- Show active sessions — what is running right now, how long it has been running, and what it is waiting for.
- Show lock chains — who is blocking whom, how long the wait has been, and what the lock type is.
- Cancel or terminate sessions — kill a specific query or session with a documented reason, logged to the audit trail.
- Run VACUUM and ANALYZE — trigger maintenance on specific tables, monitor progress, and show before/after stats.
- Run REINDEX — rebuild bloated indexes without locking the table.
- Show table and index health — bloat, dead tuples, last vacuum, last analyze, autovacuum effectiveness.
- Never touch application data — every query the server runs targets system views and catalogs only.
- Log every action — every session termination, every maintenance operation, written to an audit trail.

3.2 Non-Goals

Not in scope	Why
CRUD on application tables	The server is a DBA tool, not an application data layer.
Schema migrations (CREATE TABLE, ALTER TABLE)	Use Flyway, Liquibase, or Alembic for migrations.
Database provisioning	Use Terraform or Pulumi.
Backup management	Use pgBackRest or a managed service.
Supporting MySQL, SQLite, or other databases	PostgreSQL-specific internals are the whole point.
Automated maintenance without confirmation	VACUUM FULL and REINDEX require an explicit call with a confirmation flag. Nothing runs silently.

4. Proposed Solution

4.1 How It Connects

The server uses two PostgreSQL roles:

- Read role — granted the built-in `pg_monitor` role. Used for all observation queries against system views. Never touches application schemas.
- Ops role — granted `pg_signal_backend` (for cancelling/terminating sessions) plus table-level privileges needed to run `VACUUM` and `ANALYZE`. Still never reads application data.

Security principle: If a query would touch an application table (not a system view or catalog), the server does not run it. The only exceptions are `VACUUM` and `ANALYZE` — maintenance commands that operate on a table but do not read or return its data.

4.2 Tool Reference

Tools are grouped into five areas. Each description is one plain sentence about what it does and what you get back.

Area A — Query Performance

Query Performance		
<code>slow_queries</code>	READ	Returns the slowest queries from <code>pg_stat_statements</code> — ranked by total time, average time, or call count. Shows how long each takes, how many times it has run, and how much I/O it uses.
<code>high_variance_queries</code>	READ	Finds queries that are usually fast but occasionally very slow. High variance often means lock contention or plan instability.
<code>queries_by_io</code>	READ	Lists queries causing the most disk reads. Useful when the database is I/O bound.
<code>temp_spill_queries</code>	WARNING	Lists queries writing to temp files on disk — a sign that <code>work_mem</code> is too low for the sorts or hash joins those queries are doing.
<code>query_regression_report</code>	READ	Compares two <code>pg_stat_statements</code> snapshots and shows which queries got slower between them.
<code>explain_query</code>	READ	Runs <code>EXPLAIN</code> on a query you provide and returns the execution plan — what joins are used, how many rows the planner expects, and where the cost is concentrated.
<code>explain_summary</code>	READ	Takes raw <code>EXPLAIN</code> output and returns a plain-language description of the bottleneck and what to consider fixing.
<code>latency_percentiles</code>	READ	Returns p50, p95, and p99 execution times across all queries and per query fingerprint.

Area B — Index Health

Index Health

missing_index_candidates **RECOMMEND** Finds tables doing lots of sequential scans — the main signal that an index is missing. Shows the table size and scan count so you can prioritise.

unused_indexes **RECOMMEND** Lists indexes with zero (or near-zero) scans since the last statistics reset. Each index you drop saves write overhead on every INSERT, UPDATE, and DELETE.

duplicate_indexes **RECOMMEND** Detects indexes covering the same columns as another index (e.g. an index on user_id made redundant by one on user_id, created_at).

bloated_indexes **ANALYSIS** Estimates how much of each index is dead space. Index bloat above ~30% is worth a REINDEX.

unindexed_foreign_keys **AUDIT** Lists foreign key columns with no supporting index. These cause full table scans on the referencing table during deletes and cascades.

suggest_indexes **RECOMMEND** Cross-references slow queries, sequential scans, and sort columns to suggest specific indexes — including composite index column ordering.

index_usage_stats **READ** Returns scan counts, tuple fetches, and I/O stats per index for a given table or schema.

index_health_summary **SUMMARY** One-line health score per schema: how many indexes are missing, unused, duplicate, or bloated.

Area C — Active Sessions & Locks

Active Sessions & Locks

active_sessions **LIVE** Shows every current session: who is connected, what state they are in (active, idle, idle in transaction), what they are waiting for, and how long they have been running.

long_running_queries **WARNING** Returns sessions that have been active for longer than a threshold. These are candidates for investigation or cancellation.

idle_in_transaction **WARNING** Lists sessions stuck in the idle in transaction state beyond a threshold. These hold locks and block autovacuum.

connection_state_breakdown **READ** Aggregates sessions by state and wait event — gives a one-glance picture of what the connection pool is doing right now.

blocking_lock_tree **LIVE** Shows the full chain of who is blocking whom — each blocking session, what lock it holds, and all the sessions waiting behind it.

lock_detail **READ** For a specific table, shows all current locks held or pending: lock mode, whether granted, and which sessions are involved.

deadlock_history **AUDIT** Parses the PostgreSQL log for deadlock events and returns which queries were involved and which transaction was rolled back.

cache_hit_rate **READ** Returns buffer cache hit ratios at three levels: overall database, per table, and per index. Flags anything below 90%.

cancel_query **ACTION** Sends a cancel signal to a specific session (pg_cancel_backend) — stops the current query but keeps the connection open. Requires a reason. Logged to audit trail.

terminate_session **ACTION** Disconnects a session entirely (pg_terminate_backend). Requires a reason and confirm: true. Logged to audit trail.

terminate_idle_in_transaction **ACTION** Terminates all sessions idle in transaction beyond a specified duration. Shows a preview before acting. Requires confirm: true.

Area D — Table & Server Health

Table & Server Health

table_bloat_report **READ** Shows the dead tuple ratio per table. Tables above 10% are candidates for VACUUM; above 20% are urgent.

autovacuum_status **READ** Shows the last autovacuum and last autoanalyze timestamp per table alongside current dead tuple count. Identifies tables where autovacuum is falling behind.

autovacuum_config **READ** Shows the effective autovacuum trigger thresholds for a specific table (considering table-level overrides vs. global defaults) and how close the table is to triggering.

statistics staleness **AUDIT** Lists tables with a high number of row changes since the last ANALYZE. Stale statistics cause the query planner to make bad decisions.

schema_health **AUDIT** Checks for common schema problems: tables without primary keys, foreign keys missing indexes, and CHECK constraints that were added as NOT VALID.

table_sizes **READ** Returns total size, data size, and index size per table, and the ratio of index size to data size.

connection_utilization **READ** Shows current connections vs max_connections, broken down by state. Flags when utilisation exceeds 70%, 85%, or 95%.

server_io_stats **READ** Returns checkpoint frequency, bgwriter stats, and the ratio of buffers written by backends vs. the bgwriter — a key signal that shared_buffers may be too small.

replication_lag **READ** On a replica, shows lag in bytes and estimated seconds. Flags replicas that are falling behind.

wal_pressure **READ** Shows WAL write volume per time window. High WAL generation correlated with slow queries often points to write amplification.

Area E — Maintenance Operations

Maintenance Operations

vacuum_table **ACTION** Runs VACUUM (or VACUUM ANALYZE) on a specific table. Returns dead tuple count before and after, and the new last_vacuum timestamp. Requires confirm: true. Logged.

vacuum_full_table **ACTION** Runs VACUUM FULL — rewrites the table to reclaim all bloat. Locks the table for the duration. Requires confirm: true and acknowledged_lock: true. Logged.

analyze_table **ACTION** Runs ANALYZE on a table to refresh planner statistics. Requires confirm: true. Returns n_mod_since_analyze before and confirms the updated timestamp after.

reindex_table	ACTION	Runs REINDEX TABLE CONCURRENTLY — rebuilds all indexes on a table without locking reads or writes. Requires confirm: true. Logged.
reindex_index	ACTION	Runs REINDEX INDEX CONCURRENTLY on a single bloated index. Requires confirm: true. Logged.
vacuum_progress	READ	Shows in-flight VACUUM operations and their progress: how many heap blocks have been scanned and how many dead tuples removed.
reindex_progress	READ	Shows in-flight REINDEX operations and their current phase and block completion.
maintenance_estimate	READ	Before running VACUUM FULL or REINDEX, estimates: table size, approximate duration, lock type required, and which operations will be blocked. Run this first.
reset_table_stats	ACTION	Resets the statistics counters for a specific table so you get a clean baseline after major maintenance. Requires confirm: true. Logged.

4.3 Audit Trail

Every tool with an ACTION tag writes a structured record to the audit log: tool name, target table or session PID, actor, parameters, timestamp, and outcome (success or error). The audit log is append-only and can be queried via a dedicated resource.

4.4 Database Type Configuration

The server accepts a db_profile setting at startup: oltp, olap, mixed, or analytics_replica. This affects the thresholds used across tools — for example, an OLAP warehouse doing full scans should not flag every sequential scan as a problem, but an OLTP system probably should.

Profile	How thresholds change
oltp	Sequential scans on tables > 10,000 rows flagged. Cache hit rate threshold: 98%. Tight idle-in-transaction threshold (2 min).
olap	Sequential scans only flagged on tables > 5M rows. Cache hit rate threshold: 85% (scans expected). Looser idle-in-transaction threshold (30 min).
analytics_replica	Write-based tools (vacuum, terminate) disabled. Read-only observability only.
mixed	Balanced defaults. Appropriate for applications with both transactional and reporting workloads.

4.5 Query Library & Saved Workflows

Every tool in this server has its SQL pre-written and version-tested inside the codebase. But there is a second layer: a query library that lets operators store named queries and multi-step workflows. This is the primary mechanism for reducing API cost and preventing the LLM from generating its own SQL.

Core idea: Instead of the LLM reasoning about which tools to call and in what order across multiple API round trips, an operator pre-defines the sequence as a named workflow. The LLM calls the workflow once, gets a composite result, and never needs to chain calls manually. Tool schema overhead is paid once, not N times.

4.5.1 Named Stored Queries

Stored queries are SQL templates saved in a YAML file that ships with the server. They are callable by name via a single tool. The LLM never sees the SQL — it just calls the name and receives the result.

Stored Query Tools

- | | | |
|-----------------------------------|-------------|--|
| <code>list_saved_queries</code> | READ | Returns all available named queries with their description, parameter names, and expected result shape. The LLM calls this first to discover what queries exist. |
| <code>run_saved_query</code> | READ | Executes a named query by name with optional typed parameters (e.g. <code>table_name</code> , <code>limit</code> , <code>threshold</code>). Returns the result. The SQL is pre-written on the server — the LLM only supplies the name and parameters. |
| <code>list_saved_workflows</code> | READ | Returns all available named workflows with their description and the sequence of steps they run. |
| <code>run_workflow</code> | READ | Executes a named multi-step workflow. Runs all steps server-side in a single API call and returns a composite structured result. The LLM pays one round trip for work that would otherwise take 6–14. |

4.5.2 How Named Queries Are Defined

Queries are defined in a `saved_queries.yaml` file. Operators can add their own without touching Python code.

```
# saved_queries.yaml
- name: top_bloated_tables
  description: "Tables with highest dead tuple ratio, ordered by urgency"
  params:
    - name: limit type: int default: 20
    - name: min_pct type: float default: 0.05
  sql: |
    SELECT relname AS table, n_dead_tup, n_live_tup,
    ROUND(n_dead_tup::numeric / NULLIF(n_live_tup + n_dead_tup,0), 3) AS
    dead_ratio,
    last_autovacuum, last_autoanalyze
    FROM pg_stat_user_tables
    WHERE n_dead_tup::numeric / NULLIF(n_live_tup + n_dead_tup,0) > :min_pct
    ORDER BY dead_ratio DESC LIMIT :limit
- name: slow_callers
  description: "Top 10 queries by total time – the costliest workload by absolute impact"
```

```
sql: |
  SELECT query, calls, total_exec_time, mean_exec_time, rows
  FROM   pg_stat_statements ORDER BY total_exec_time DESC LIMIT 10
```

4.5.3 How Saved Workflows Are Defined

Workflows are sequences of named steps. Each step is either a built-in tool or a stored query. The server runs them all in one pass and returns a single structured result.

```
# workflows.yaml
- name: weekly_health_review
  description: "Full weekly DBA health check - bloat, indexes, slow queries, cache"
  steps:
    - tool: slow_queries          params: { limit: 5 }
    - tool: missing_index_candidates
    - tool: unused_indexes
    - query: top_bloated_tables  params: { limit: 10 }
    - tool: autovacuum_status
    - tool: cache_hit_rate
    - tool: connection_utilization
- name: slow_query_triage
  description: "Quick incident triage: active sessions, locks, and top slow queries"
  steps:
    - tool: active_sessions
    - tool: blocking_lock_tree
    - tool: slow_queries          params: { limit: 5 }
    - tool: connection_utilization
```

4.5.4 Result Caching

Every stored query and every built-in tool result is cached with a TTL. The cache is keyed by (query_name, params_hash). If a result is fresh, it is returned immediately without touching the database. This also means that in a workflow, repeated queries across steps do not re-hit the database.

Query type	Default TTL	Rationale
pg_stat_statements (slow queries, IO)	30 seconds	Statistics update roughly every second. 30s gives meaningful data without hammering the stats collector.
pg_stat_activity (active sessions, locks)	10 seconds	Session state changes fast. Cache must be short or it misses blocking events.
pg_stat_user_tables (bloat, vacuum)	120 seconds	Table stats change slowly. 2 minutes is fine for most use cases.
Index stats (usage, bloat estimates)	300 seconds	Rarely change within a session. 5-minute TTL is safe.
Server metrics (connections, bgwriter)	60 seconds	Changes at a medium pace. 1-minute TTL balances freshness and cost.

Saved query results (custom queries)	Configurable	Set per query in saved_queries.yaml. Default: 60 seconds.
Workflow results	Min TTL of all steps	The workflow cache inherits the shortest TTL of its component steps.

4.5.5 Suggested Built-in Workflows

Workflow name	Steps included
weekly_health_review	slow_queries, missing_index_candidates, unused_indexes, top_bloated_tables, autovacuum_status, cache_hit_rate, connection_utilization
slow_query_triage	active_sessions, blocking_lock_tree, slow_queries, connection_utilization
index_audit	missing_index_candidates, unused_indexes, duplicate_indexes, bloated_indexes, unindexed_foreign_keys, index_health_summary
pre_deploy_check	schema_health, statistics_staleness, connection_utilization, table_bloat_report
incident_response	active_sessions, blocking_lock_tree, long_running_queries, idle_in_transaction, slow_queries
vacuum_candidate_report	top_bloated_tables, autovacuum_status, statistics_staleness, autovacuum_config

5. Cost Analysis & Feasibility

Using an MCP server means every interaction routes through an LLM API. Token costs are real. This section gives honest estimates of what running this server will cost and whether those costs are justifiable.

5.1 What Drives Token Costs

Every time an MCP client (Claude, another agent) calls a tool, an API request is made. That request contains:

- Tool schema definitions — the names, descriptions, and parameter schemas of all tools loaded for the session. This is a fixed overhead on every single API call.
- Conversation history — everything said and returned so far in the session. This grows with each turn.
- Tool results — the data the server returns from PostgreSQL (query rows, plan output, session lists). These are usually the biggest variable.
- Model output — the LLM response interpreting results and deciding next steps.

Key insight: Tool schema overhead is unavoidable and accumulates. With 35 tools at ~75 tokens each, every API call starts with ~2,600 tokens of overhead before a single word of conversation. This is the dominant cost driver for short sessions.

5.2 Token Estimates Per Component

Component	Low estimate	High estimate	Notes
Tool schema (all 35 tools, per API call)	2,400 tokens	2,800 tokens	Fixed per call. Cannot be reduced without loading fewer tools.
slow_queries result (top 10)	500 tokens	1,200 tokens	Depends on query text length.
active_sessions result (20 sessions)	700 tokens	1,500 tokens	Grows with session count.
blocking_lock_tree result	300 tokens	800 tokens	Deeper chains = more tokens.
explain_query output (medium query)	600 tokens	3,500 tokens	Complex plans with many nodes can be very large.
table_bloat_report (50 tables)	800 tokens	2,500 tokens	Linear with table count.
missing_index_candidates (10 results)	400 tokens	900 tokens	Reasonably compact.
suggest_indexes (5 recommendations)	400 tokens	900 tokens	Compact with rationale.
cache_hit_rate (20 tables)	500 tokens	1,200 tokens	Grows with table count.
Model response per turn	200 tokens	800 tokens	Short for tool calls, longer for final summaries.

5.3 Cost Per Session — Scenario Estimates

Prices use Claude Sonnet (\$3 per million input tokens, \$15 per million output tokens). These are illustrative — costs vary by model and provider.

Scenario A: Quick check ("What are my slowest queries?")

1–2 tool calls. User asks one question, agent calls slow_queries, returns a summary.

Item	Input tokens	Output tokens	Cost
Tool schemas (×2 API calls)	5,200	—	—

User message + history	150	—	—
slow_queries result	800	—	—
Model responses	—	600	—
Total	~6,150	~600	~\$0.03

Scenario B: Slow query investigation ("Why is this query slow, and what should I fix?")

4–6 tool calls. Agent calls slow_queries → explain_query → missing_index_candidates → suggest_indexes, then synthesises recommendations.

Item	Input tokens	Output tokens	Cost
Tool schemas (×6 API calls)	15,600	—	—
User messages + growing history	800	—	—
Tool results (4 calls combined)	4,500	—	—
Model responses	—	2,000	—
Total	~20,900	~2,000	~\$0.09

Scenario C: Weekly health review (full audit)

10–14 tool calls. Covers bloat, vacuum, indexes, slow queries, cache hit rates, and produces a maintenance script.

Item	Input tokens	Output tokens	Cost
Tool schemas (×14 API calls)	36,400	—	—
User messages + growing history	1,500	—	—
Tool results (14 calls combined)	12,000	—	—
Model responses	—	4,000	—
Total	~49,900	~4,000	~\$0.21

Scenario D: Incident investigation ("Database is slow right now, find it and fix it")

15–20 tool calls. Active sessions, lock trees, running VACUUM or cancelling sessions, followed by index analysis.

Item	Input tokens	Output tokens	Cost
Tool schemas (×20 API calls)	52,000	—	—
User messages + growing history	2,500	—	—
Tool results (20 calls combined)	18,000	—	—

Model responses	—	6,000	—
Total	~72,500	~6,000	~\$0.31

5.4 Monthly Cost Projections

Usage pattern	Sessions/month	Avg session cost	Monthly cost	Annual cost
Automated weekly review only	4	\$0.21	\$0.84	\$10
Light — one dev, occasional checks	30	\$0.06	\$1.80	\$22
Moderate — small team, regular use	120	\$0.09	\$10.80	\$130
Heavy — team of 5, daily investigations	400	\$0.12	\$48	\$576
Very heavy — ops team, incident response	1,000	\$0.15	\$150	\$1,800

Context: A single hour of DBA consulting runs \$100–250. At moderate usage, the entire monthly LLM cost is less than 10 minutes of DBA time. Even at very heavy usage, the annual cost is well below the salary cost of a part-time DBA. The economics are strongly favourable.

5.5 Token Cost Risks

Risk	Detail and mitigation
Large databases (500+ tables) return massive tool results	table_bloat_report on 500 tables returns ~12,000 tokens per call. Mitigate: paginate results (return top 50 by dead ratio, not all 500). Make result limits configurable per tool.
Complex EXPLAIN ANALYZE output can be enormous	A deeply nested plan with parallel workers on a complex join can exceed 8,000 tokens. Mitigate: cap explain output at 3,000 tokens by default. Offer a verbose=true flag for full output when needed.
Long sessions where context accumulates	A 20-turn investigation session can accumulate 80,000+ input tokens just from growing conversation history. Mitigate: encourage agents to summarise and start fresh sessions rather than extending indefinitely.
Tool schema overhead scales with tool count	35 tools × ~75 tokens × 20 API calls = 52,500 tokens per session in schema overhead alone. Mitigate: support tool filtering so a user doing only index analysis loads only index tools (~10 tools = ~750 tokens overhead).
Automated agents calling tools in a loop	An agent that iterates over all tables individually (one vacuum_table call per table) will accumulate context rapidly. Mitigate: schema-wide operations and server-side batching with a single tool call.

5.6 Ways to Reduce Token Usage

Optimisation	Expected saving
Tool filtering — load only the tools relevant to the current task	20–60% reduction in schema overhead. A session doing only index analysis uses ~750 tokens of schema overhead instead of 2,600.
Paginate all list-type results to top 20–50 rows by default	30–50% reduction in tool result size on large databases. Most useful answers come from the highest-ranked results.
Truncate query text in results to 150 characters	15–25% reduction in <code>slow_queries</code> and <code>active_sessions</code> result sizes.
Compact JSON output format (no pretty-printing, abbreviated keys)	10–20% reduction in all tool result sizes.
Start a fresh session for each distinct investigation rather than extending one session	Prevents context accumulation from dominating cost in long sessions. A fresh session resets conversation history to zero.
Cache <code>pg_stat_statements</code> results for 30 seconds	No token saving, but reduces database load when the same tool is called repeatedly. Token cost still applies per API call.

5.7 Feasibility Verdict

Dimension	Assessment
Technically feasible	Yes. FastMCP + psycopg3 is proven (crystaldba uses the same stack). PostgreSQL system views are stable and well-documented. No novel infrastructure required.
Economically feasible	Yes. At realistic usage volumes, monthly LLM costs are negligible compared to the DBA time they save. Even at very heavy usage, annual cost is well under \$2,000.
Context window feasible	Yes for typical use. Claude Sonnet has a 200K token context window. A full incident investigation (20 tool calls) uses roughly 80,000 tokens — comfortably within limits. Risk exists only for extremely large databases with unpaginated results.
Latency feasible	Yes. Observation tools complete in under 5 seconds. The main latency source is LLM API response time (1–5 seconds per turn), not database query time.
Risk: Large database result sizes	Manageable with pagination. A 1,000-table database returning full bloat reports untruncated is the main failure mode. Addressed by making result limits a first-class configuration.
Risk: Runaway agent loops	An agent that calls tools in a tight loop without summarising can burn tokens rapidly. Addressed by designing prompts to summarise and conclude rather than iterate endlessly.

5.8 Cost Impact of the Query Library

Saved workflows are the single most effective cost reduction. By collapsing N tool calls into one API call, the fixed schema overhead is paid once instead of N times. This section shows the before/after delta for each built-in workflow.

How workflows reduce cost

Every API call carries ~2,600 tokens of tool schema overhead regardless of what the call does. A 7-step workflow without the query library costs 7 API calls × 2,600 tokens = 18,200 tokens in schema overhead alone. With `run_workflow`, that collapses to 1 API call × 2,600 tokens = 2,600 tokens.

Workflow	Steps	API calls without	API calls with	Token saving	Approx cost saving
weekly_health_review	7	7	1	~15,600 input tokens	~\$0.16 per run → \$0.03
slow_query_triage	4	4	1	~7,800 input tokens	~\$0.09 per run → \$0.03
index_audit	6	6	1	~13,000 input tokens	~\$0.13 per run → \$0.03
pre_deploy_check	4	4	1	~7,800 input tokens	~\$0.07 per run → \$0.02
incident_response	5	5	1	~10,400 input tokens	~\$0.11 per run → \$0.03
vacuum_candidate_report	4	4	1	~7,800 input tokens	~\$0.07 per run → \$0.02

Biggest win: The `weekly_health_review` workflow saves ~82% of its token cost compared to calling tools individually. At 52 runs per year, that is \$8.32 saved annually from one workflow alone. Across all workflows at moderate team usage, the query library reduces overall API spend by 40–60%.

Updated monthly cost projections (with query library)

Usage pattern	Without library	With library	Monthly saving	Annual saving
Automated weekly review only	\$0.84	\$0.16	\$0.68	\$8
Light — occasional checks	\$1.80	\$0.90	\$0.90	\$11
Moderate — small team	\$10.80	\$4.80	\$6.00	\$72
Heavy — team of 5	\$48	\$20	\$28	\$336
Very heavy — ops team	\$150	\$65	\$85	\$1,020

Additional cost reduction: result caching

Caching prevents repeated database hits when the same query is called multiple times within a session. Token cost is not reduced by caching (the LLM API call still happens each time), but database load is reduced. The bigger benefit is that within a workflow, shared data (e.g. pg_stat_user_tables queried by both bloat and vacuum tools) is only fetched once.

Cache benefit	Applies when
Single DB fetch for shared data in workflows	bloat report and autovacuum status both read pg_stat_user_tables — cached after first step, reused in second
Rapid repeat calls within a session	Agent calls slow_queries twice in 20 seconds — second call served from cache, no DB hit
Multiple agents querying same server	Two different Claude sessions query the same server within the TTL window — both get cached response

Caching does not reduce API token cost. To reduce API cost, use workflows. To reduce database load, caching handles it. They serve different purposes.

6. Alternatives Considered

6.1 Extend crystalldba/postgres-mcp

The most obvious option — crystalldba is well-designed, focused, and has 2,000+ stars. We considered contributing session management and maintenance tools to it.

Why not	Development has slowed significantly. The project has 25 open issues with no recent activity. Contributing a major feature expansion to a project with uncertain maintenance continuity is risky for our team. We also need db_profile-aware thresholds and an audit trail that the existing architecture does not accommodate cleanly.
---------	---

6.2 Use neverinfamous/postgres-mcp with filtering

It covers health and index analysis and has a tool-filtering system. We could configure a minimal subset.



Why not	Its 248-tool surface includes CRUD on application tables, schema migrations, backup operations, and more. Even with filtering, the codebase is built around full database access, not the principle of never touching application data. The security model is fundamentally different from what we need.
---------	--

6.3 Build on top of pgEdge MCP

pgEdge's custom YAML tool system is an interesting extension point.

Why not	pgEdge is primarily a natural language query tool, not a DBA observability platform. Adding the full set of tools we need via YAML/PL/pgSQL would essentially mean rebuilding everything on top of a framework that was not designed for it.
---------	--

6.4 Add a single execute_sql tool and let the LLM write its own pg_stat queries

Simpler to build — just expose one generic SQL execution tool.

Why not	LLMs make mistakes with system view queries — wrong column names, version-specific differences, joining pg_stat_activity to pg_locks incorrectly. Named tools with tested, version-aware SQL are significantly more reliable. A generic tool also cannot enforce the "system views only" security constraint.
---------	---

7. Trade-offs & Risks

7.1 Risk Register

Risk	Likelihood	Impact	Mitigation
Agent terminates the wrong session — pid reuse between observation and action	Low	High	Session tools check (pid, backend_start) pair, not pid alone. Stale pid confirmation prevents acting on a recycled pid.
VACUUM FULL locks a table during business hours	Medium	High	maintenance_estimate must be called first and shows the lock type and estimated duration. vacuum_full_table requires acknowledged_lock: true. Document that this should only run in low-traffic windows.

Querying <code>pg_stat_statements</code> adds load to an already stressed database	Medium	Medium	Tool responses are cached with a configurable TTL (default 30s). A <code>statement_timeout</code> guards all auditor queries. The connection pool for the auditor is separate from the application pool.
<code>pg_stat_statements</code> not enabled — performance tools fail silently	Medium	Medium	Server checks for the extension at startup and returns a clear error with installation instructions if absent. Tools degrade gracefully.
<code>terminate_idle_in_transaction</code> bulk-terminates legitimate long-running batch jobs	Medium	High	Tool returns a dry-run preview before acting. Supports <code>application_name</code> filters to exclude known batch processes. Requires <code>confirm: true</code> .
LLM recommends a harmful index on a high-write table based on stale data	Medium	Medium	Index recommendations include estimated write overhead alongside read benefit. All recommendations are advisory — the server never creates indexes.
VACUUM FULL and REINDEX run concurrently, saturating I/O	Low	Medium	Server enforces a max concurrent maintenance operations limit (default: 2). Additional requests are queued.
Active sessions view exposes query text containing sensitive parameter values	Low	High	Configurable <code>query_text_mode</code> : full truncate hide. Default is truncate at 200 characters.

7.2 Key Trade-offs

Read-only observation vs. maintenance actions

The server is not purely read-only — it can cancel sessions and run VACUUM. This is a deliberate expansion beyond what `crystalldba` offers. The trade-off is added surface area for mistakes. We mitigate this by requiring explicit confirmation flags, showing previews before bulk operations, and logging everything. All maintenance actions are still limited to the system layer — no application data is modified.

Named tools vs. a generic SQL interface

Named tools (`slow_queries`, `vacuum_table`) are less flexible than a generic `execute_sql` tool but significantly safer. The LLM cannot accidentally query an application table, cannot call `pg_stat_statements_reset()`, and cannot run a query that takes 10 minutes and starves the connection pool. Every named tool has a tested, version-aware SQL implementation and a bounded statement timeout.

8. Open Questions

#	Question	Context
OQ-1	Should VACUUM FULL be available at all, given it fully locks the table?	Some teams will never accept that risk in their MCP toolset. Consider making it opt-in at server configuration time, disabled by default. Owner: DBA.
OQ-2	What is the right db_profile taxonomy and who decides the thresholds for each?	OLTP vs OLAP vs mixed affects almost every threshold in the tool set. Needs agreement from DBAs and Platform Eng before implementation. Blocking for most tools.
OQ-3	How should terminate_session and cancel_query be protected in a multi-team deployment?	If two teams share one MCP server, one team could terminate the other's sessions. Options: per-tool RBAC, role restrictions, or require a secondary approval. Owner: SecOps.
OQ-4	Should explain_query run EXPLAIN ANALYZE in production, or only EXPLAIN?	ANALYZE executes the query, which adds load. A production_safe_mode config flag that disables ANALYZE by default might be the right approach. Owner: DBA.
OQ-5	What should the cache_hit_rate threshold be, and should it vary by db_profile?	Connected to OQ-2. An OLAP warehouse always has lower cache hit rates. Threshold must be profile-aware to avoid constant false positives. Owner: DBA.
OQ-6	Should the server snapshot pg_stat_statements automatically for regression reports, or require manual snapshots?	Auto-snapshotting adds statefulness to the server. Manual snapshots are simpler but require the agent to manage them. Owner: Backend Eng.
OQ-7	What happens to an in-flight VACUUM if the MCP server restarts?	PostgreSQL will finish the VACUUM — the server just loses its monitoring handle. Should the server detect orphaned in-flight operations on startup? Owner: Backend Eng.
OQ-8	Should maintenance operations (vacuum, reindex) be runnable schema-wide, or only per-table?	Schema-wide VACUUM on a large database could take hours. Per-table is safer but requires iteration. A batched mode with progress checks is a possible middle ground. Owner: DBA + Backend Eng.
OQ-9	Minimum supported PostgreSQL version?	pg_stat_io is PG16+. pg_stat_statements columns changed in PG13. Some tools need conditional SQL per version. Recommend PG13 minimum. Owner: Platform Eng.
OQ-10	Should GUC tuning recommendations (work_mem, shared_buffers) account for managed services like RDS or Aurora?	Some parameters cannot be changed on managed services (huge_pages, shared_preload_libraries). Server needs cloud provider awareness. Owner: Platform Eng.

Priority: OQ-2 (db_profile thresholds) and OQ-3 (multi-team session termination access control) must be resolved before the first tool group is implemented. They affect the design of almost everything else.

9. Implementation Roadmap

Phase	Timeline	What gets built
1 — Foundation	Weeks 1–3	Server scaffold, two-role setup, <code>slow_queries</code> , <code>latency_percentiles</code> , <code>explain_query</code> , <code>explain_summary</code> , <code>active_sessions</code> , <code>connection_state_breakdown</code> , <code>cache_hit_rate</code> , <code>connection_utilization</code>
2 — Index & Schema	Weeks 4–5	<code>missing_index_candidates</code> , <code>unused_indexes</code> , <code>duplicate_indexes</code> , <code>bloated_indexes</code> , <code>unindexed_foreign_keys</code> , <code>suggest_indexes</code> , <code>table_bloat_report</code> , <code>schema_health</code> , <code>statisticsstaleness</code>
3 — Maintenance	Weeks 6–7	<code>vacuum_table</code> , <code>analyze_table</code> , <code>reindex_table</code> , <code>reindex_index</code> , <code>vacuum_progress</code> , <code>reindex_progress</code> , <code>maintenance_estimate</code> , <code>reset_table_stats</code> — full audit logging for all ACTION tools
4 — Sessions & Locks	Weeks 8–9	<code>blocking_lock_tree</code> , <code>lock_detail</code> , <code>deadlock_history</code> , <code>long_running_queries</code> , <code>idle_in_transaction</code> , <code>cancel_query</code> , <code>terminate_session</code> , <code>terminate_idle_in_transaction</code> , <code>vacuum_full_table</code>
5 — Server Health	Weeks 10–11	<code>autovacuum_status</code> , <code>autovacuum_config</code> , <code>server_io_stats</code> , <code>wal_pressure</code> , <code>replication_lag</code> , <code>table_sizes</code> , <code>high_variance_queries</code> , <code>queries_by_io</code> , <code>temp_spill_queries</code> , <code>query_regression_report</code>
6 — Hardening	Weeks 12–14	PostgreSQL version compatibility (PG13–17), <code>db_profile</code> threshold calibration, statement timeout tuning, load testing, security review, documentation